NPU-Native Neural Networks on AMD XDNA 2

Built with IRON/MLIR-AIE — target device: AMD Ryzen AI 9 HX 370 (Strix Point)

Hardware-software co-design for close-to-metal neural network inference

Abstract

This document is a compact tutorial and research note about co-designing a neural network for the AMD XDNA 2 NPU. The working example is deliberately simple: a 32-layer residual MLP for MNIST, with one learnable weight matrix per compute tile and activations flowing through the full 32-tile array in a snake. The point is not that MNIST is hard, nor that this particular design reaches the hardware's peak throughput — it does not. The point is that it is small enough to understand end to end, yet rich enough to expose the real constraints that matter on this hardware: local SRAM, MMUL-friendly data layouts, explicit FIFOs, and the cost of host–device orchestration.

1 Introduction

Modern NPUs are most interesting when they stop looking like "small GPUs" and start looking like what they are: spatial dataflow machines. On AMD XDNA 2, the programmer sees a 2D array of compute tiles, each with local SRAM, a VLIW + SIMD core, a matrix-multiply unit, and explicit data-movement primitives. That makes this hardware unusually transparent. It also means that a model that looks natural in PyTorch may look awkward once you try to map it directly onto the array.

This repository takes the hardware seriously. Instead of beginning with a large model and then asking how to squeeze it onto the device, we begin with the tile array and ask: what is the cleanest learnable network that maps onto it almost literally? The answer in the current main branch is a 32-layer residual MLP for MNIST:

$$h_{\ell+1} = \mathrm{ReLU}(h_\ell W_\ell) + h_\ell, \qquad \ell = 0, \ldots, 31.$$

Each of the 32 compute tiles stores exactly one residual weight matrix. In the main public $H = 160$ split, the host CPU still runs the input embedding, while the NPU runs all 32 hidden layers and the final classification head in one streamed service.

The result is now a family of didactic but fully working examples:

- the main $H = 160$ end-to-end path reaches about 488,597 images/s, or roughly 3.5× over pure CPU inference on the same host,
- the resident $H = 160$, $B = 8$ residual body reaches about 1.46 million images/s wall-clock at `stream_depth=32`, or about 2.39 TFLOPS,
- a narrower throughput-oriented $H = 96$, $B = 56$ variant reaches about 97.3% validation accuracy on a 50k/10k train/validation split, about 739k images/s wall-clock in the current host-visible service, and about 3.88 million images/s if only NPU kernel wait time is counted.

These numbers are intentionally modest relative to the NPU's peak. The current paper is therefore not a claim that this is the fastest possible use of XDNA 2. It is a clear case study in co-design: how the shape of a network changes once local memory, tiling, and routing become first-class design constraints. Older, more experimental iterations of the project are preserved in `logbook.md` and on the `experimental` branch; this whitepaper focuses only on the current `resmlp/` implementation.

Mental model	CPU / GPU intuition	XDNA 2 intuition
Compute	A few large kernels over shared memory	Many small tile-local kernels connected by explicit streams
State	Parameters and activations live in DRAM / cache / HBM	Parameters should fit in tile SRAM whenever possible
Data movement	Often implicit (cache hierarchy, kernel launch runtime)	Explicit (DMA, ObjectFIFOs, placement, routing)
Architecture choice	Mostly driven by model quality	Driven by both model quality and physical layout

Table 1: The whitepaper assumes the reader is comfortable with neural networks, but new to spatial dataflow hardware. The central shift is from "launch bigger kernels" to "shape the network so each tile has a clean local job."

Property	Value
Compute array	32 compute tiles = 8 columns × 4 compute rows
Per compute tile	~64 KB local data memory + tile-local program memory
MemTile row	8 memory tiles, 512 KB each
Compute primitives	VLIW + SIMD core and bf16/BFP16-friendly MMUL instructions
Peak headline numbers	~25 TFLOPS (bf16), 50 TOPS (int8) [4, 5]
Software stack	IRON / MLIR-AIE / XRT [1, 2, 3]

Table 2: Only a few hardware facts matter for this paper. The important one is not the marketing TOPS number, but the very small per-tile SRAM budget.

2 The hardware model we program against

2.1 Why spatial dataflow changes the design problem

A CPU or GPU programmer usually thinks in terms of a few large kernels operating against a shared memory system. XDNA 2 rewards a different mental model: keep small state local to each tile, move only what must move, and make the routing pattern of the computation explicit.

For this reason, the first design question in this project is not "what is the best architecture for MNIST?" but rather "what architecture maps cleanly onto a 32-tile array with ~64 KB of SRAM per tile?" That shift is what makes the project interesting.

2.2 XDNA 2 at a glance

Figure 1 shows the physical layout we target. The Ryzen AI 9 HX 370 exposes 32 compute tiles arranged as 8 columns by 4 compute rows. Below them sit 8 MemTiles, and below those 8 shim tiles that connect the array to host-visible memory and runtime control. The main branch of this repository uses all 32 compute tiles.

2.3 One compute tile, from a programmer's point of view

Figure 2 is a logical view of one compute tile. It is not a physical floorplan. It is the subset of the architecture that matters when writing code with IRON and MLIR-AIE.

Three remarks are worth keeping in mind.

1. The most valuable resource is not peak arithmetic throughput; it is the ability to keep parameters local in SRAM.
2. The MMUL datapath likes blocked 8×8 layouts, which is why this project insists on dimensions divisible by 8 and uses explicit tiling on the host side.
3. Data movement is explicit. Activations do not "just appear" on the next tile. They move because FIFOs and DMA descriptors were created for them.

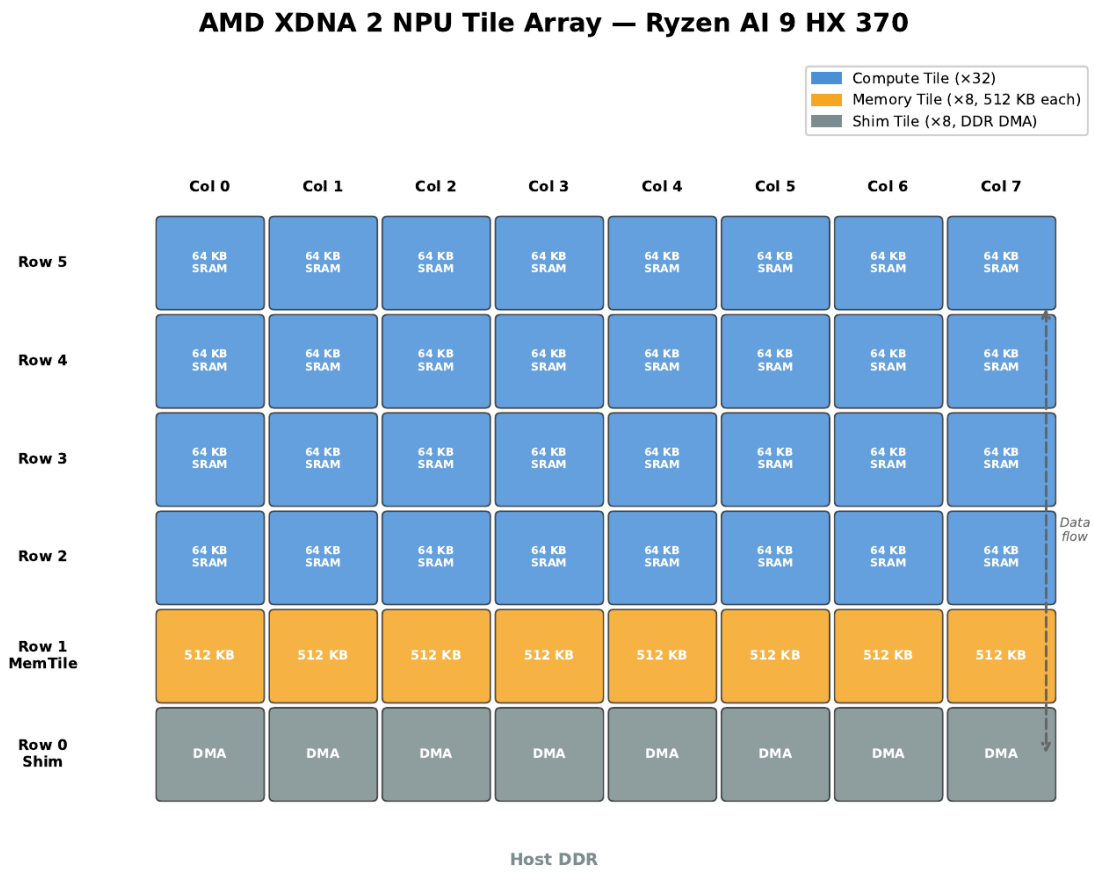

Figure 1: Physical layout of the AMD XDNA 2 NPU used here. The project uses all 32 compute tiles (8 columns × 4 rows). MemTiles provide additional on-chip memory and routing support; shim tiles connect the array to host memory and runtime control.

2.4 The constraints that actually shape the design

The current `resmlp/` implementation is driven by a short list of very practical constraints, shown in Table 3.

3 A network that matches the array

3.1 Model overview

The current model is intentionally plain. In the main $H = 160$ streaming path, the CPU flattens a 28×28 MNIST image and projects it to a 160-dimensional hidden state. On the NPU, we apply 32 copies of the same layer shape, each with its own weights, and the final tile also projects the last hidden state to ten logits. A later subsection revisits the same 32-layer template at a narrower, higher-batch operating point ($H = 96$, $B = 56$).

For the main $H = 160$ streaming path, this split is not arbitrary. The embedding stays on the CPU because a one-tile resident $784 \to 160$ projection is not SRAM-feasible, while the hidden stack and the small $160 \to 10$ head fit comfortably on-array. The hidden stack is the repetitive part: it is where a tile array is easiest to use well, because the same buffer shape recurs 32 times.

The PyTorch model in `resmlp/model.py` is therefore almost identical to the hardware view:

```
class ResidualLinear(nn.Module):
```

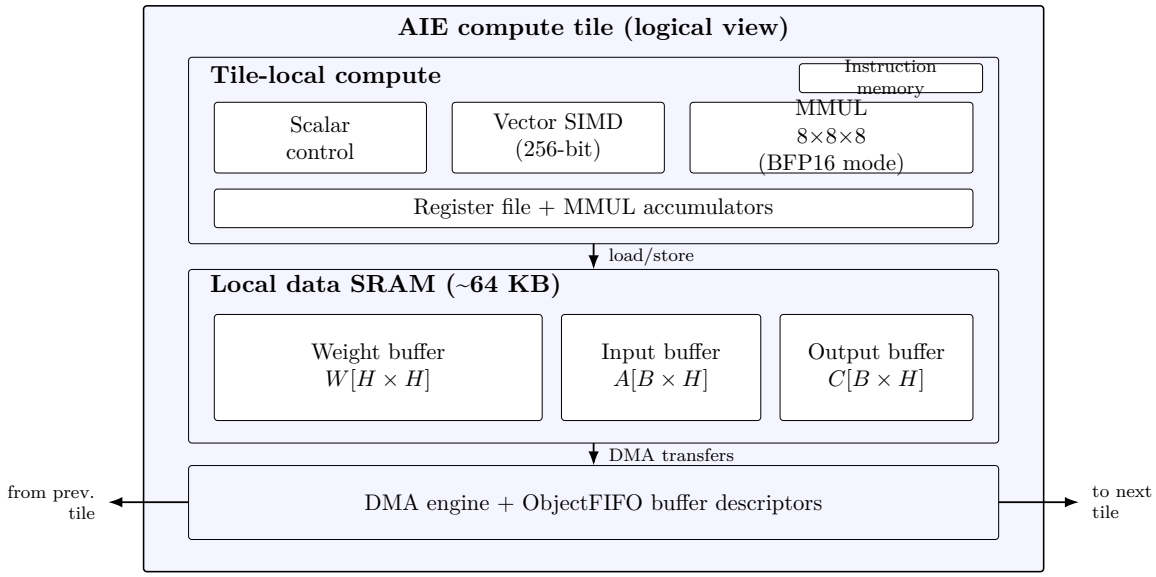

Figure 2: Logical view of one XDNA 2 compute tile: local SRAM, MMUL-backed compute, and explicit DMA / FIFO data movement.

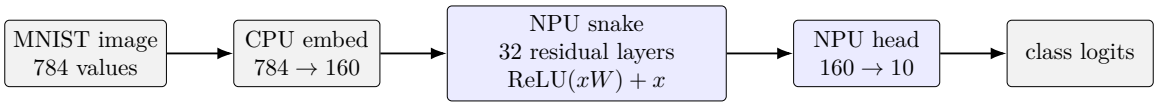

Figure 3: Host/NPU split in the main $H = 160$ streaming path: CPU embedding, 32 NPU residual layers, then an NPU classification head on the final tile.

```
    def forward(self, x):
        return torch.relu(x @ self.weight) + x

class ResMLP(nn.Module):
    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = self.embed(x)
        for layer in self.layers:
            x = layer(x)
        return self.head(x)
```

That symmetry matters. The main reason this whitepaper is easy to write at all is that the conceptual model and the deployed model are so close to one another.

3.2 Snake routing through all 32 tiles

The compute tiles are not used as eight independent columns in the current main branch. Instead, they are chained into one long serpentine path. The activation enters the first tile, flows through all 32 layers, and leaves the last tile. This is the cleanest way to make "one weight matrix per tile" literally true.

In code, this ordering is generated by a tiny helper:

```
def snake_tile_order(num_cols):
    tiles = []
    for col in range(num_cols):
        rows = range(2, 6) if col % 2 == 0 else range(5, 1, -1)
        for row in rows:
            tiles.append((col, row))
```

Constraint	Why it matters	Choice in this project
~64 KB per tile	Weight matrix, activations, and scratch space must fit locally	Choose $H = 160$: 50 KB weights + 5 KB activations + headroom
8×8 MMUL-friendly layout	Host tensors must be packed in blocked layout; dimensions should be multiples of 8	Choose $B = 8$ and $H = 160$; use `to_tiled()` / `from_tiled()`
Explicit routing	Every inter-tile handoff has to be representable as a FIFO / route	Use a single snake of plain `ObjectFifo`s across all 32 tiles
Invocation overhead	Small calls waste hardware; fixed overhead dominates when the batch is tiny	Accept this for clarity; frame `B=8` as pedagogical, not peak-throughput
Current runtime semantics	Finite XRT calls still shape the schedule, but inference no longer needs per-call weight DMA	Use checkpoint-specific inference XCLBINs with embedded tile-local weights and repeated shim DMA boundary tasks; validated deep streaming up to `stream_depth=32` on the full $H = 160$ path

Table 3: This is the minimal constraint set needed to understand the rest of the document. Anything else belongs in an appendix, the code, or the logbook.

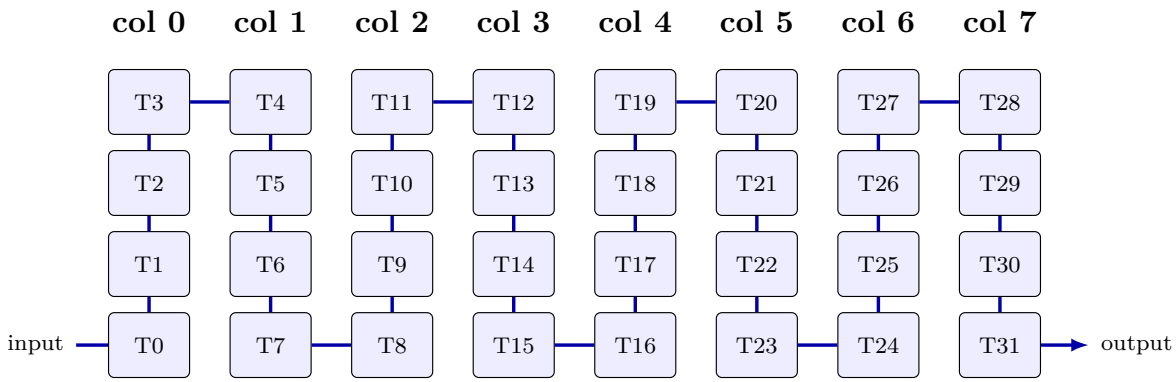

Figure 4: The 32-tile snake used in `resmlp/design.py`. Even-numbered columns are traversed bottom-to-top; odd-numbered columns top-to-bottom. Each node is one residual layer with its own weight matrix.

```
    return tiles
```

That function is deceptively simple. It is the point where the network architecture stops being abstract and starts matching the physical device.

3.3 Why $H = 160$ and $B = 8$

The hidden dimension $H = 160$ is chosen by SRAM, not by theory. A single tile must hold one 160×160 bf16 weight matrix plus the input and output activation buffers for a batch of 8 samples. Figure 5 shows the budget.

Numerically:

- weight matrix: $160 \times 160 \times 2 = 51{,}200$ bytes ≈ 50.0 KB,
- input activations: $8 \times 160 \times 2 = 2{,}560$ bytes,
- output activations: $8 \times 160 \times 2 = 2{,}560$ bytes.

This lands at roughly 55 KB before stack and miscellaneous overhead, leaving comfortable headroom under the tile's local memory limit.

The batch size $B = 8$ follows from the MMUL granularity. The current kernel uses 8×8×8

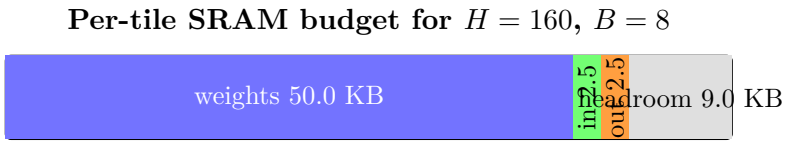

Figure 5: The key design choice in one picture. The weight matrix dominates the budget; activations are small enough that one tile can hold one full layer comfortably.

blocking and expects dimensions divisible by 8. Choosing $B = 8$ is therefore the smallest clean batch that does not waste the MMUL shape. It is emphatically not the throughput-maximizing choice. It is the smallest choice that keeps the mapping simple and the tutorial honest.

3.4 BFP16 and tiled host layouts

The main branch compiles the kernel with `-DAIE_API_EMULATE_BFLOAT16_MMUL_WITH_BFP16`. In practice this means that host-side weights and activations are arranged into blocked 8×8 layouts, and the MMUL path uses a BFP16-style execution mode that aligns well with that blocking. The helper functions `to_tiled()` and `from_tiled()` in `resmlp/__init__.py` perform the required data reordering.

This choice trades a small amount of numerical fidelity for a much cleaner and more uniform hardware mapping. In the test harness, the difference between the NPU path and a float32 reference compounds across 32 layers into noticeable per-element mismatch, but it does not meaningfully degrade classification accuracy.

4 From network to NPU program

4.1 What one tile computes

The tile-local kernel in `aie_kernels/matmul_relu_skip.cc` implements the whole layer in two steps: a blocked matrix multiply with ReLU, followed by a vectorized residual add.

```
extern "C" void matmul_relu_skip_bf16(bfloat16 *a, bfloat16 *w, bfloat16 *c)
{
    static_assert(DIM_M % 8 == 0, "Batch␣must␣be␣multiple␣of␣8");
    static_assert(DIM_K % 8 == 0, "Input␣dim␣must␣be␣multiple␣of␣8");
    static_assert(DIM_N % 8 == 0, "Output␣dim␣must␣be␣multiple␣of␣8");

    ::aie::set_rounding(aie::rounding_mode::floor);
    matmul_relu<bfloat16, (DIM_M / 8), (DIM_K / 8), (DIM_N / 8)>(a, w, c);
    residual_add(a, c);
}
```

Two details matter here.

1. The current kernel is deliberately simple. It uses a 1×1 output blocking pattern rather than the more aggressive 2×2 expansion used in older throughput-oriented experiments.
2. The residual connection is explicit in the kernel. This keeps the NPU path numerically close to the PyTorch model instead of forcing the host to perform an extra add between layers.

4.2 How IRON describes the snake

The IRON design in `resmlp/design.py` mirrors the conceptual picture almost literally:

- one activation FIFO into the first tile,
- one activation FIFO out of the last tile,

Buffer	Meaning	Size per batch
Input activation	one $B \times H$ hidden-state batch entering tile 0	2.5 KB
Output activation	one $B \times H$ batch leaving tile 31	2.5 KB
Weight buffer	all 32 tiled $H \times H$ matrices packed together	1.56 MB

Table 4: What the current runtime moves for each batch. The current implementation is weight-stationary during a call, but not across calls: the full weight buffer is written again for every batch.

- one activation FIFO between every pair of consecutive snake stages,
- one weight FIFO per column, split in the MemTile into four tile-local weight streams,
- one worker per compute tile.

The worker body itself is tiny:

```
def worker_fn(of_in, of_out, of_w, kern):
    x = of_in.acquire(1)
    y = of_out.acquire(1)
    w = of_w.acquire(1)
    kern(x, w, y)
    of_w.release(1)
    of_in.release(1)
    of_out.release(1)
```

This is one of the nicest properties of the current implementation. There is no large orchestration layer hidden elsewhere. The dataflow graph is already the program.

An important subtlety is that the code uses plain `ObjectFifo`s for the activation chain, even across column boundaries. The compiler handles the required inter-column routing through the memory infrastructure; the Python code does not manually spell out every routing hop.

4.3 What moves during one inference call

The end-to-end inference path in `resmlp/infer.py` is:

1. load a trained PyTorch checkpoint,
2. run the CPU embedding layer,
3. convert the hidden activations to tiled bf16 layout,
4. write input activations and packed weights to the NPU buffers,
5. execute the NPU runlist,
6. read back the final hidden state and apply the CPU classification head.

The sizes are instructive.

This weight rewrite cost per batch explains the modest measured performance. The model is structurally "NPU-native": each layer maps directly to one physical tile. But the current runtime still pays host-side setup and buffer-write costs every batch. This is not a bug; it is simply where the current implementation stops.

5 Results and what they mean

5.1 Accuracy and speed

The accuracy result is the easy part: the hardware path matches the software model closely enough that the NPU version classifies MNIST just as well as the CPU baseline. That gives us confidence that the mapping is correct.

The speed result needs more interpretation. A batch of 8 through 32 hidden layers performs roughly

$$32 \times (2BH^2 + BH) \approx 13.15 \text{ million operations}$$

Metric	Value
Model parameters	946K
CPU test accuracy	97.1%
NPU test accuracy	97.16%
NPU stage latency in `infer.py`	0.33 ms per batch of 8
NPU stage throughput in `infer.py`	24,114 images/s
Device-side benchmark in `test.py`	55 GFLOPS for the full 32-tile snake

Table 5: The current main-branch results. The two performance numbers answer slightly different questions: `infer.py` includes host-side buffer writes for each batch, while the device-side benchmark isolates the NPU run itself more directly.

for $B = 8$ and $H = 160$, counting both the matrix multiply and the residual add. That is not much work for a device with a 25-TFLOP bf16 headline. The design is therefore dominated by overheads: invocation cost, buffer writes, and the simple fact that $B = 8$ was chosen for clarity, not for saturation.

5.2 Why the current design is far from peak throughput

The right way to read the current performance is not "the NPU is slow." The right way to read it is "this particular example is optimized for transparency." Three factors dominate.

1. **Small batch.** $B = 8$ is the smallest neat MMUL-aligned batch, but it leaves little work to amortize fixed costs.
2. **Per-batch weight writes.** The current runtime writes the entire 1.56 MB weight buffer every batch, even though the weights do not change.
3. **Simple kernel scheduling.** The main branch uses the simplest clean kernel and routing strategy, not the most aggressively throughput-tuned one.

This is why the paper should not be read as an argument that "the best use of XDNA 2 is MNIST with $B = 8$." It is instead an argument that a clean end-to-end example can make the co-design problem legible.

5.3 Why the example is still valuable

Despite the modest utilization, the current design teaches several things very cleanly.

- A neural network can map directly to the physical tile graph with very little abstraction gap.
- The most important design question on this hardware is often SRAM budget, not parameter count in the abstract.
- A simple CPU/NPU split — CPU for irregular edges, NPU for the repetitive core — is often the cleanest way to proceed.
- Explicit data movement clarifies the true cost model: what stays local, what streams, and what the host still has to do.

That is precisely why this repository pivoted away from more fragmented earlier iterations and toward the current `resmlp/` example.

6 Conclusion and next directions

The cleanest story in this repository is not "we reached peak TOPS." It is "we built a neural network whose structure is dictated by the hardware, and we can explain every part of that mapping." That is already a useful result.

For future work, the next steps are straightforward:

- keep weights resident across calls instead of rewriting them every batch,

- increase batch size to better amortize invocation overhead,
- explore int8 variants,
- move to a task that is more representative than MNIST once the mapping is fully understood.

The older experimental history remains valuable — especially the throughput benchmarks and language-model work — but it belongs in `logbook.md` and the `experimental` branch. The main whitepaper should stay sharp: one device, one mapping, one working example, fully explained.

A Current full-NPU training implementation

The main text explains the cleaner mapping that the current whitepaper is built around: CPU embedding, a 32-tile residual snake on the NPU, then a CPU classification head. The code in this repository now also contains a more ambitious training path in `resmlp/train_npu.py`. That path moves the entire MNIST model onto the NPU for one training call: embedding, residual stack, classification head, loss, backward pass, and SGD updates.

This appendix documents that implementation as it exists today. The important point is not that every systems problem is solved. It is that the training computation itself now lives on the array. The remaining rough edges are mostly runtime-state and persistence problems, not missing backpropagation math.

A.1 Tile layout and layer mapping

The full training graph still uses all 32 compute tiles, but not in the same role assignment as the inference snake from the main text. In `resmlp/training_full_design.py`, the 32 tiles are partitioned as shown in Table 6.

Tiles	Role	What runs there
tile 0	embedding	`embed_forward_bf16` and `embed_backward_bf16`
tiles 1–30	hidden stack	one residual layer per tile: `matmul_relu_skip_bf16` forward and `residual_backward_and_update_bf16` backward/update
tile 31	head + loss	`head_forward_loss_bf16` and `head_backward_bf16`

Table 6: The current full-NPU training decomposition. The full path uses one embed tile, thirty residual tiles, and one head tile.

This means that `full-npu` is a 30-residual-layer training design, not the older 32-residual-layer inference snake described earlier. The difference is structural, not cosmetic: the two end tiles are now used for the irregular edges of the network instead of being forced into the residual pattern.

A.2 What one training call does on the array

One invocation of the full training program performs the whole forward and backward path on device.

1. The host streams a raw 8×784 MNIST batch into the embed tile.
2. The embed tile applies the $784 \to 32$ projection.
3. The activation flows through 30 residual tiles in snake order.
4. The head tile computes logits, softmax-cross-entropy quantities, class predictions, and the tiled `d_logits` buffer.
5. The backward pass starts at the head tile and propagates gradients back through the residual chain to the embed tile.

6. Each stage applies its own SGD update in place.

The backward kernels are the main novelty. The head kernel in `aie_kernels/head_backward.cc` computes

$$dY_{\text{hidden}} = d\text{logits}\, W_{\text{head}}^T$$

and then applies

$$W_{\text{head}} \leftarrow W_{\text{head}} - \eta\, Y_{\text{hidden}}^T d\text{logits}.$$

The residual kernel in `aie_kernels/residual_backward.cc` performs both gradient propagation through the skip-ReLU layer and the outer-product SGD update of the local 32×32 weight matrix. The embed kernel in `aie_kernels/embed_backward.cc` does the same for the 784×32 projection matrix.

In other words, the appendix-level claim is strong but precise: SGD is not simulated on the host after an NPU forward pass. The weight updates themselves are executed inside the AIE kernels.

A.3 How IRON wires the training graph

The IRON graph in `resmlp/training_full_design.py` is the clearest place to see how the training step is organized.

- `embed_wt` and `head_wt` are direct weight FIFOs.
- Residual weights enter as eight per-column FIFOs and are split in the MemTiles into 3 or 4 tile-local weight streams, matching the snake occupancy of each column.
- Activations form one forward chain of `ObjectFifo`s.
- Gradients form one backward chain of `ObjectFifo`s.
- Each residual tile stores a compact checkpoint buffer containing its input activation and ReLU mask, so the backward phase can reconstruct the local gradient update.
- A small `done_fifo` marks backward completion, and `head_wt_out` drains the updated head weights back to host-visible memory.

One subtle but important detail is the embed backward phase. The embed tile does not keep the full raw 8×784 batch as a forward checkpoint in local SRAM. Instead, the host re-streams `x_raw` during the backward phase. This is a good example of the actual programming style on the array: whenever a buffer is too awkward to keep resident, the design should make that data movement explicit.

A.4 What works today, and what does not

The current implementation is good enough to support the central claim of this appendix: the full network, the backward pass, and the SGD update rules can all run on the NPU in one training call. It is not yet a fully resident training loop across many batches.

The stable path in `run_full_npu_batch()` still uses a fresh `AIEContext` for every batch. That workaround remains necessary because reusing the same hardware context corrupts later invocations. The important refinement is that the wrapper no longer needs a full `DefaultNPURuntime.cleanup(` on every call. Evicting only the cached hardware contexts is enough to restore correct execution while keeping the instruction-buffer cache hot. Long runs still need a small amount of host-side garbage collection to release the invalidated contexts. In the current code, collecting every 256 full-NPU batches is enough to stay stable well past the two-thousand-context mark where a pure no-GC wrapper starts to fail.

In the current stable configuration, host-visible persistence works reliably for `head_wt` only. The embed and residual weights are updated in tile memory during a call, but the system does not yet drain them back in a form that can be reused safely across calls. Earlier attempts either exceeded SRAM, failed routing, or exhausted DMA descriptor resources.

This changes the performance picture substantially. The NPU work itself still takes about 5 ms for the $B = 8$, $H = 32$ training design, but the wrapper around it is now only a small extra cost instead of a 250 ms teardown penalty. A limited 500-batch probe now completes in about 3.7 s while preserving the same learning behavior as the older fresh-runtime wrapper. So the present result should be read as both a correctness milestone and evidence that the runtime overhead is no longer the dominant bottleneck for this path.

Even with that limitation, the path is no longer a dummy stub. In the current repository, the stable full-NPU training mode learns on MNIST instead of remaining at chance. After adding the periodic-GC safeguard, a full two-epoch MNIST run completes on hardware. In one representative run, epoch 0 takes about 77.6 s, epoch 1 about 125.3 s, and the final host-side accuracies are about 44.9% on the training set and 45.6% on the test set. The remaining gap to a cleaner throughput story comes from host-side fresh-context orchestration and missing full weight persistence, not from missing forward, backward, or SGD math on the array.

B One-column simple convnet bring-up

To separate "training math works on the array" from the harder residual-state and persistence problems, the repository now also contains a deliberately small alternative path in `simplecnn/`. This design uses only one compute column and four trainable tiles:

- tile 0: a $1 \to 4$ convolution, 3×3, stride 2, producing $14 \times 14 \times 4$,
- tile 1: a $4 \to 8$ convolution, 3×3, stride 2, producing $7 \times 7 \times 8$,
- tile 2: an $8 \to 16$ convolution, 3×3, stride 2, producing $4 \times 4 \times 16$, followed by global average pooling,
- tile 3: a linear $16 \to 10$ classifier with a tiny top-1 hinge update.

This path started as a compile-oriented simplification, but it still needed a surprising amount of correctness work before the metrics meant anything. The generic conv training kernel overflowed tile program memory, so the final design uses three fixed-shape kernels (`conv1_train.cc`, `conv2_train.cc`, `conv3_train.cc`). More importantly, the model and the kernels had to agree on several non-obvious conventions:

- conv weights are packed as `[out, ky, kx, in]`, not PyTorch's native `[out, in, ky, kx]`,
- the pooled 8×16 buffer must stay row-major all the way into the classifier head, not be silently reinterpreted as an 8×8 tiled matrix,
- generated MLIR / XCLBIN / instruction artifacts must fingerprint kernel source changes as well as the Python design files, otherwise stale binaries can survive apparently real rebuilds,
- conv weight drainback needs explicit `wait=True`; without it, the head weights drained correctly while the conv weights arrived on the host as zeros.

After these fixes, one-batch NPU updates matched a bf16-quantized CPU replay of the same kernels closely, and the batch predictions matched exactly. That made the remaining limitation much clearer: the one-column convnet is now a parity-checked, stable full-NPU training path, but it is not yet a good MNIST model.

That original baseline is mixed but useful. The design is stable across 10 epochs with no numeric failure or runtime corruption, and the measured NPU call itself is small: about 4.7–5.3 ms. Throughput is also respectable for such a tiny pedagogical design, roughly 1.3–1.4k images/s. But the learning signal remains weak: the best test accuracy in this run is 10.77% at epoch 0, and the final test accuracy is 10.16% at epoch 9, which is effectively chance on MNIST.

Epoch	Train acc	Test acc	Time (s)	Img/s	ms/call
0	0.1114	0.1077	46.2	1298	5.3
1	0.1027	0.1033	42.3	1417	4.7
2	0.1031	0.1024	42.3	1420	4.7
3	0.1036	0.1023	42.4	1414	4.7
4	0.1034	0.1061	42.1	1425	4.7
5	0.1084	0.1065	42.4	1414	4.7
6	0.1084	0.1047	42.8	1401	4.7
7	0.1045	0.1023	42.0	1429	4.7
8	0.1031	0.1014	41.9	1433	4.7
9	0.1029	0.1016	41.5	1446	4.7

Table 7: Ten-epoch MNIST run for the original batch-oriented `simplecnn` baseline: batch size 8, one compute column, four trainable tiles, top-1 hinge head, NPU SGD step $\eta = 0.05$, and one NPU call per batch.

B.1 Resident-window follow-up

After the 10-epoch bring-up, the next question was whether the one-column path could follow the same broad strategy used in AMD's official one-column streaming examples. The closer analogue is the edge-detect example: one task pipeline over a single compute column, using neighboring-tile memories and ping-pong-style buffering to reduce dead time between kernels. The scaled color-threshold example is a useful secondary reference, but it is primarily a MemTile split/concat design for data-parallel fanout and fanin, not a layer-parallel training pipeline.

The practical adaptation for `simplecnn` was therefore at the host/runtime boundary first. The current CLI now groups minibatches into resident windows, keeps one shared host/device `weights` BO live for the whole window, and uses per-batch taps to address slices of larger host buffers. Crucially, the tile-side `ObjectFifo` payloads remain one-batch wide: making the tile FIFOs themselves window-sized immediately overflowed local SRAM, so the only scalable pattern is "large host buffers, small tile buffers, repeated taps". The current conservative default is `-window-batches 5`.

Window batches	Train batches	NPU calls	Img/s	ms/call
1	10	10	371	19.5
5	10	2	470	78.5
5	100	20	456	84.7

Table 8: Resident-window `simplecnn` probes on hardware. The first row is the same resident host path forced back to `window_batches=1`; the later rows show the validated default `window_batches=5`.

This is real progress, but it is not yet the fully overlapped edge-detect-style win. The current workers still execute forward and backward for one batch before advancing to the next microbatch, and the graph still uses depth-1 FIFOs. So the gain above comes mainly from reducing host launches and host-visible weight traffic, not from a true multi-batch in-array pipeline. Stable windows are currently verified through five microbatches; `window_batches=8` already compiles but still times out at runtime.

This makes the current interpretation straightforward. The one-column convnet is not yet a useful MNIST baseline, but it *is* a useful systems baseline. It shows that a much simpler four-tile training graph can now be compiled, validated against a quantized CPU replay, run stably for many epochs, and now also execute short resident windows with measurably lower host overhead. The remaining work is now about model quality and true in-array overlap, not whether the basic forward/backward/update path is functioning.

C Deep streaming residual inference

The most encouraging recent result is that the repository now has a genuinely useful inference-only streaming path for the original 32-tile residual snake. The key simplification is to stop treating inference like training. For training, weights and ReLU state need to move in and out of the array and the host has to preserve backward-facing context. For inference, none of that is necessary. Each tile can simply hold its own fixed checkpoint weight matrix in SRAM, consume one activation microbatch, emit one activation microbatch, and do so repeatedly.

This gives a design that is still easy to describe:

- the CPU keeps the image flattening, embedding, and classifier head,
- the NPU keeps the 32-layer residual body,
- each compute tile stores one blocked 160×160 bf16 weight matrix as a compile-time initialized local `Buffer`,
- the host-facing runtime contract shrinks to just `input` and `output` buffers,
- repeated calls reuse the same instructions and the same embedded weights, so the host no longer re-sends the residual body parameters every batch.

C.1 What changed in the implementation

Two details mattered much more than they first appeared.

First, inference needed its own kernel entrypoint. The original `matmul_relu_skip.cc` kernel had been extended for training so that it could also write a ReLU mask for the backward pass. That is correct for the training graph, but inference only wants the three logical tensors `(a, w, c)`. Reusing the five-argument training entrypoint in an inference graph silently scribbled mask data into unrelated buffers and made deep streaming look unstable long before the dataflow itself was the real problem. The final solution is a dedicated three-argument `matmul_relu_skip_infer_bf16` wrapper for inference while preserving the mask-writing entrypoint for training.

Second, artifact naming had to become more disciplined. Once the same source file can produce both training and inference entrypoints across several hidden sizes, short names like "`resmlp_kernel.a`" or "`resmlp_streaming_kernel.a`" are no longer safe. The current operators fingerprint the relevant Python design files, kernel source, and, for streaming inference, the packed checkpoint weights themselves. That avoids the particularly confusing failure mode where an apparently fresh rebuild links a stale object compiled for a different hidden size or ABI.

C.2 From resident weights to deep streaming

With weights embedded, the first gain is easy to understand: even at `stream_depth=1`, the design is already much faster than the old one-shot path because the host no longer pushes all 32 layer weights through DDR and the MemTiles on every call. The next step is to let one call carry several microbatches. In the current implementation, `stream_depth` means exactly that: how many $B = 8$ activation microbatches are fed through the snake before control returns to the host.

The first implementation of this idea still hit an avoidable runtime ceiling. Even though the array-side workers were happy to loop over many microbatches, the host runtime issued one fill task and one drain task per microbatch. That kept the boundary FIFOs small, but it meant that deeper chunks stressed the shim-task schedule before they stressed the arithmetic.

The fix was to keep the tile-local FIFOs exactly as small as before (`depth=2` for `stream_depth > 1`) and instead change the boundary runtime contract. The current design uses a single repeated shim DMA fill and a single repeated shim DMA drain per call, driven by a custom `TensorAccessPattern` whose leading size is `stream_depth`. In practice, that means one runtime

task can replay the same $B \times H$ microbatch transfer many times, so the first and last compute tiles never have to allocate a giant activation window in SRAM just to reach a deeper steady state.

This is still not a fully free-running conveyor belt. Control still returns to the host after a finite chunk, and a true indefinitely running service would need a more persistent runtime abstraction. But inside that chunk, the array now does behave like the desired layered pipeline: while later tiles are working on older microbatches, earlier tiles can already start the next ones. The measured throughput scaling confirms that this overlap is real all the way to `stream_depth=32`.

Mode	Images / call	Img/s	ms/call	Speedup vs one-shot
One-shot residual inference	8	35,026	0.228	1.0×
Repeated-BD streaming, depth 1	8	70,771	0.113	2.0×
Repeated-BD streaming, depth 2	16	137,510	0.116	3.9×
Repeated-BD streaming, depth 4	32	260,881	0.123	7.4×
Repeated-BD streaming, depth 8	64	478,327	0.134	13.7×
Repeated-BD streaming, depth 16	128	898,220	0.143	25.6×
Repeated-BD streaming, depth 32	256	1,458,411	0.176	41.6×

Table 9: Full $H = 160$, $B = 8$, 8-column / 32-tile residual body on hardware. The one-shot baseline and all streaming rows use the same arithmetic workload; only the host/runtime contract changes.

These numbers matter for two reasons. First, they show that resident weights alone are already worthwhile: depth 1 is about twice as fast as the old one-shot path. Second, the near-linear gain from depth 1 all the way to depth 32 shows that multiple microbatches really are overlapping in the array rather than merely waiting in host memory. The current full-path sweep reaches about 1.46 million images/s wall-clock throughput at depth 32, which corresponds to about 2.39 TFLOPS for the residual body. In a longer one-million-image benchmark, the same depth-32 configuration sustains about 1.51 million images/s wall-clock and about 1.59 million images/s if only kernel wait time is counted, or roughly 2.47 / 2.60 TFLOPS.

These figures intentionally isolate the resident residual body. The public Python inference service still has to do endpoint work on the host — at minimum layout conversion, and in the current $H = 160$ path also the image embedding — so end-to-end MNIST throughput is lower than the body-only upper bound above.

C.3 What depth 32 means — and what it does not

At steady state, `stream_depth=32` finally realizes the intuitive conveyor-belt picture for this 32-layer snake: each compute tile can be working on a different $B = 8$ microbatch at the same time. That is the occupancy target the earlier finite-task runtime could not safely reach.

What this does *not* mean is that the whole MNIST inference graph is now on the NPU. The current public $H = 160$ streaming path still keeps image flattening and embedding on the CPU; the residual body and classifier head now run on the NPU. That is a better end-to-end boundary than the earlier CPU-head split, but it still does not eliminate host work entirely: the residual-body measurements above are best read as the throughput ceiling of the resident snake itself, not as the wall-clock throughput of the full host-visible inference service.

C.4 End-to-end speedups versus pure CPU inference

To keep the throughput story honest, it helps to compare the full host-visible inference path against a plain PyTorch CPU baseline on the same machine. Table 10 includes the CPU-side endpoint work that still exists today: CPU embedding for the main $H = 160$ split, and the CPU head for the narrower $H = 32$ embed-on-tile-0 prototype.

Path	Hidden dim	Img/s	Speedup vs pure CPU
Pure CPU eager inference	160	140,658	1.0×
CPU embed + 32-tile NPU body + NPU head	160	488,597	3.5×
Pure CPU eager inference	32	824,670	1.0×
NPU embed + 31-tile NPU body + CPU head	32	413,664	0.50×

Table 10: End-to-end inference throughput on the same host. The pure CPU rows use the best measured PyTorch thread count on this machine (12 threads for $H = 160$, 2 threads for $H = 32$). The NPU rows include the host-side boundary work that still remains in the current Python service path.

The conclusion is encouraging, but also clarifying. For the original $H = 160$ design, the NPU path now delivers a real end-to-end gain of roughly 3.5× over pure CPU inference on this host. Moving the head onto the last tile also buys a modest but measurable gain over the earlier CPU-head split. The resident 32-tile snake is therefore not just a pretty internal benchmark; it does translate into a visible wall-clock speedup even after host-side boundary work is included.

The $H = 32$ prototype tells a different story. Moving the first layer onto tile 0 is architecturally clean and it does work reliably at `stream_depth=32`, but it is not yet a throughput win. The first tile now spends real time on the $784 \to 32$ projection, and the service still has to untile the final hidden state and run the classifier head on the CPU. In other words, endpoint migration is not automatically synonymous with higher throughput; it only helps if the new on-array stage is cheaper than the host work it replaces.

C.5 A throughput-oriented $H = 96$, $B = 56$ point

The same 32-layer residual template can also be pushed toward a more throughput-friendly point on the SRAM frontier by narrowing the hidden state and growing the microbatch. The best current example uses a $784 \to 96$ embedding, 32 residual layers of shape 96×96, and a $96 \to 10$ head. That model has $784 \cdot 96 + 96 = 75{,}360$ parameters in the embed, $32 \cdot 96 \cdot 96 = 294{,}912$ in the residual body, and $96 \cdot 10 + 10 = 970$ in the head, for a total of 371,242 trainable parameters. One forward pass costs about 742,272 FLOPs/image if only the matrix multiplies are counted, or about 748,522 scalar floating-point operations/image if bias adds, ReLUs, and residual adds are included.

On a 50k/10k MNIST train/validation split, a simple CPU-trained checkpoint reaches 97.27% validation accuracy, and the streamed inference service reproduces 97.25% through the NPU residual body. Using $B = 56$ and `stream_depth=32`, the repeated-image service benchmark processes about 739,473 images/s wall-clock in the current host-visible path and about 3.88 million images/s if only NPU kernel wait time is counted, corresponding to about 0.55 / 2.91 TFLOPS if the whole network forward pass is charged to the measurement. For this shape the public fused logits tail is not yet stable, so the present benchmark uses the correct body-only streamer with CPU embedding and CPU head in the Python service loop.

We do *not* claim this is a universal literature record. Direct MNIST throughput comparisons are surprisingly scarce and often differ in model family, quantization, batching, or whether host-side work is included at all. Public benchmark collections such as QuTiBench and GPU/TPU batch-size studies report MNIST inference regimes in the hundreds of thousands of images/s on very different systems [6, 7]. Our measured wall-clock figure therefore appears to be at least at the high end of the public MNIST throughput numbers we could find, but it should be read as a strong data point rather than an apples-to-apples claim of beating every published result.

Taken together, these results are still strong while keeping the design conceptually modest:

- no backward pass,
- no host weight BO for the residual body,

- one checkpoint-specific inference XCLBIN,
- one explicit scalar knob, `stream_depth`, controlling the amount of in-array overlap,
- one repeated shim DMA fill and one repeated shim DMA drain per call, rather than one host DMA task per microbatch.

That is exactly the kind of systems win worth keeping in the main branch: a clear design and a measured throughput story. The remaining limitation is now a much better one to have: not "can depth 32 work at all?", but "how much more of the surrounding graph should move on-array, and how much closer to peak can this runtime contract go?".

D Compact glossary

Term	Meaning in this project
AIE tile	One compute tile of the XDNA 2 array: local memory + local compute + DMA connectivity
MemTile	The memory tile that sits between shim tiles and compute tiles; useful for weight distribution and routing
Shim tile	The interface tile that connects the array to host-visible memory and runtime infrastructure
ObjectFIFO	IRON abstraction for explicit data channels between producers and consumers
MMUL	Tile-local matrix-multiply datapath used by the AIE kernel intrinsics
BFP16	Execution mode used here to make 8×8×8 blocking clean and efficient
XCLBIN	Compiled device artifact loaded by XRT
XRT	Runtime that launches the compiled design on the NPU

Table 11: Only the terms that are repeatedly needed in the main text are kept here.

E Toolchain in one page

The software pipeline in this repository is short enough to summarize directly:

```
aie_kernels/matmul_relu_skip.cc
    -> LLVM-AIE object
    -> kernel archive

resmlp/design.py
    -> Python-generated MLIR-AIE module

MLIR-AIE + aiecc
    -> .xclbin + instruction binary

XRT runtime
    -> buffer setup + runlist execution on the NPU
```

The IRON project and the MLIR-AIE ecosystem are the main external dependencies for the hardware side of this repository [1, 2]. The Linux AMD NPU documentation is also useful for understanding the runtime and driver-facing view of the device [3].

References

[1] AMD, *IRON repository.*
https://github.com/amd/IRON

[2] Xilinx / AMD, *MLIR-AIE programming guide.*
https://github.com/Xilinx/mlir-aie/tree/main/programming_guide

[3] Linux kernel documentation, *AMD NPU / amdxdna.*
https://docs.kernel.org/next/accel/amdxdna/amdnpu.html

[4] AMD, *AI Engine documentation (programming model and architecture).*
https://docs.amd.com/r/en-US/ug1273-versal-acap-design/AI-Engine

[5] *AMD XDNA overview.*
https://en.wikipedia.org/wiki/AMD_XDNA

[6] M. Blott et al., *QuTiBench MNIST classification benchmark collection.*
https://rcl-lab.github.io/QutibenchWeb/mnist/rooflines/performance%20prediction/2020/03/25/MNIST_Classification.html

[7] Y. Gordienko et al., *Batch Size Influence on Performance of Graphic and Tensor Processing Units During Training and Inference Phases.*
https://link.springer.com/chapter/10.1007/978-3-030-16621-2_61